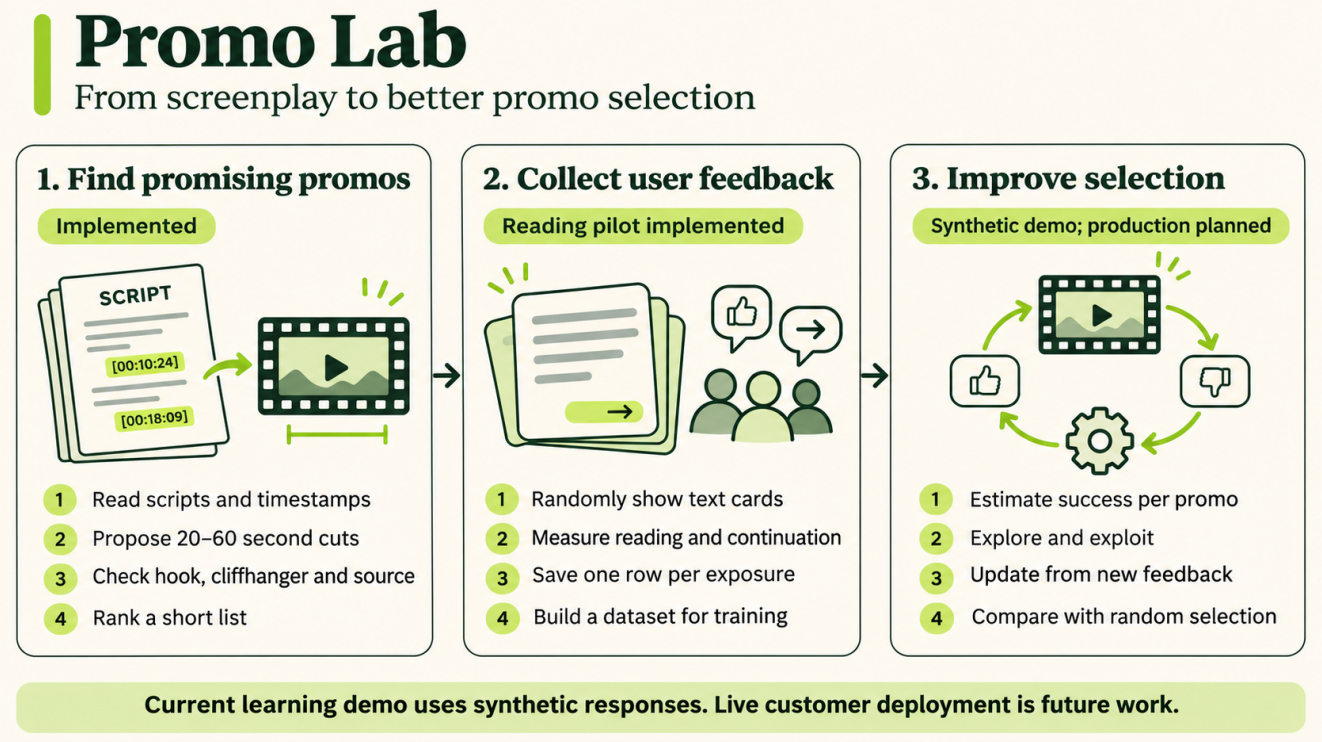
Promo Lab

I chose Find the Promos because editors can verify each cut against the script, and viewers can show us which cuts make them want to continue. My approach uses an LLM to find promising promos, then learns from viewer behavior.

Assumptions. Timestamps are episode-relative, and dialogue, action and camera tracks can overlap. A 20-60 second promo can join up to three segments in chronological order. With scripts as input, the pilot uses reading as a proxy for watching.



My three-stage pipeline

1. LLM candidate retrieval. Parse scripts into timestamped events. Generate cuts from script boundaries and ask the LLM for more using event IDs. Code checks source, duration, order and dialogue boundaries. A second LLM pass scores the hook, clarity, stakes, progression, cliffhanger and cut quality. Group overlaps and target the top 1% of distinct valid cuts, subject to score thresholds.

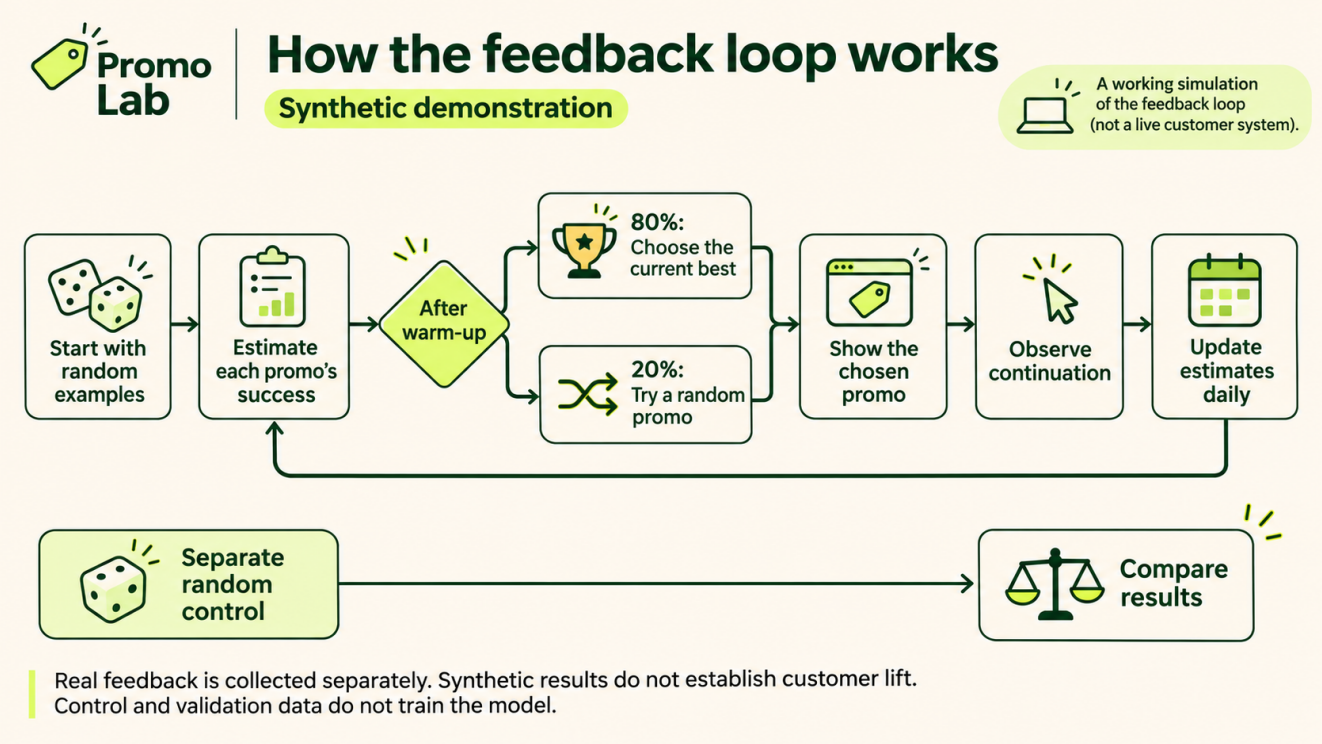
2. Internal survey in a simulated feed. Ask employees to browse a reel-style feed with one random promo among six unrelated stories. The text-card prototype logs one row per exposure with consent. A positive label requires a continuation click, 15 active seconds and 75% of the continuation text seen. These behavior labels would train the first model.

3. Epsilon-greedy selection and online training. In a live pilot, start with the model trained on internal feedback. Explore heavily at first. Once stable, show the highest-scoring promo 80% of the time and choose randomly 20%. Log each outcome and selection probability. Update daily, giving older feedback less weight. Compare qualified continuation against a separate random control. The prototype simulates this loop with per-promo estimates and held-out validation.

How it would scale

Process episodes in parallel, cache by source, prompt and model, and prune similar cuts before critique. Track uncached token use, cost and coverage per show. Collect feedback on real footage, evaluate on later data and compare the updated policy with a randomized control before rollout.

Feedback loop and results



From 50 episodes to 6 promos

I ran the tool on the supplied scripts. Of 16,023 possible clips, 14,758 passed the script and timing checks. Grouping near-duplicates left 553 distinct clips. The 1% target gave 6 slots after rounding up. The AI successfully scored 48 finalists, at most one per episode. The tool recommended the 6 highest-ranked clips that met the scoring rules.

How I checked the recommendations

I checked that the timestamps matched the script, the clips lasted 20-60 seconds and no lines were cut short. For comparison, I took roughly 30 seconds from each episode's first spoken line. 46 of those 50 clips passed, and 4 cut a line short. Of the 6 recommendations, 6 passed and 0 cut a line short. Viewer interest still needs to be measured in the feed.

Where it fails

Observed AI error. The model suggested a 128.65-second clip. The tool rejected it because a promo must last 20-60 seconds.

The model can mistake a confusing ending for suspense. Scoring one finalist per episode can miss a stronger clip. An editor still needs to watch the footage to check performances and transitions.

AI tools used

Codex assisted implementation, tests and writing. The reported run used qwen/qwen3.8-27b. OpenRouter Nemotron was checked on the original public example. ImageGen created both diagrams.